Java Multithreading & Concurrency

Java Backend Interview — Short Notes

1. Thread Basics

1.1 Process vs Thread

	Process	Thread
Definition	Independent program in execution with its own memory space	Lightweight unit of execution within a process
Memory	Own heap, stack, code, data segments	Shares heap with other threads in same process; own stack
Communication	IPC (pipes, sockets, shared memory) — expensive	Shared heap — fast but needs synchronisation
Creation	Expensive — OS allocates new memory space	Cheap — just a new stack and PC register
Crash impact	Crash isolated to that process	One thread crash can bring down entire process
Context switch	Expensive — full state save/restore	Faster — shares address space

1.2 Thread Lifecycle / States

State	Description	Transitions
NEW	Thread created but start() not yet called	→ RUNNABLE via start()
RUNNABLE	Ready to run or currently running. Scheduler decides execution	→ BLOCKED/WAITING/TIMED_WAITING/TERMINATED
BLOCKED	Waiting to acquire a monitor lock (synchronized block/method)	→ RUNNABLE when lock acquired
WAITING	Indefinitely waiting — called wait(), join(), LockSupport.park()	→ RUNNABLE via notify()/notifyAll()/interrupt()
TIMED_WAITING	Waiting with timeout — sleep(n), wait(n), join(n), parkNanos()	→ RUNNABLE when timeout expires or notified
TERMINATED	run() completed normally or threw uncaught exception	Cannot restart — create new Thread

1.3 Creating Threads — Four Ways

1. Extend Thread class

```
class MyThread extends Thread {
@Override public void run() { System.out.println("Thread: "+getName()); }
}
new MyThread().start(); // start() creates new thread; run() directly = same thread
```

2. Implement Runnable (Preferred — separates task from thread management)

```
Runnable task = () -> System.out.println("Running: "+Thread.currentThread().getName());
new Thread(task).start();
```

3. Implement Callable (Returns result, can throw checked exception)

```
Callable task = () -> { return 42; };
```

```
Future future = executor.submit(task);
Integer result = future.get(); // blocks until done
```

4. Via ExecutorService (Preferred in production — thread reuse)

```
ExecutorService exec = Executors.newFixedThreadPool(4);
exec.submit(() -> doWork());
exec.shutdown();
```

✔ Prefer Runnable/Callable with ExecutorService. Avoid manually creating Thread in production code

1.4 Key Thread Methods

Method	Description	Notes
start()	Create new OS thread and call run() in it	Calling run() directly = no new thread
run()	Entry point — override for task logic	Called by JVM on new thread via start()
sleep(millis)	Pause current thread for given ms — TIMED_WAITING	Does NOT release lock. InterruptedException
join()	Current thread waits for target thread to finish	join(timeout) — wait at most timeout ms
interrupt()	Set interrupt flag; unblocks sleeping/waiting thread	Check isInterrupted(). Sleeping throws InterruptedException
isInterrupted()	Check interrupt flag without clearing it	Thread.interrupted() — checks AND clears flag
yield()	Hint to scheduler to give up CPU — non-binding	Rarely used. No guarantee
setDaemon(true)	Mark as daemon — JVM exits when only daemons remain	Must call before start(). GC thread is daemon
isDaemon()	True if daemon thread	—
setPriority(n)	Set priority 1-10 (MIN=1, NORM=5, MAX=10)	Hint only — OS decides actual scheduling
getName()/setName(str)	Get/set thread name — useful for debugging	Default: Thread-0, Thread-1, etc.
currentThread()	Static — returns reference to currently running thread	Thread.currentThread().getName()
getState()	Return current Thread.State enum value	NEW/RUNNABLE/BLOCKED/WAITING/TIMED_WAITING/TERMINATED
Thread.sleep(0)	Releases CPU to other threads of same priority	yield() hint, sleep(0) is slightly stronger

2. Synchronisation

2.1 synchronized Keyword

• Ensures mutual exclusion — only one thread executes synchronized code at a time

• Each object has an intrinsic lock (monitor). synchronized acquires the lock on entry, releases on exit (even on exception)

Synchronized Method — locks on 'this'

```
public synchronized void increment() { count++; }
// equivalent to: public void increment() { synchronized(this) { count++; } }
```

Synchronized Static Method — locks on Class object

```
public static synchronized void staticMethod() { ... }
// locks on Counter.class — different lock from instance lock
```

Synchronized Block — fine-grained, preferred

```
private final Object lock = new Object(); // dedicated lock object
public void method() {
// non-critical work here
synchronized(lock) {
count++; // only critical section locked
}
}
```

• **Reentrant** — same thread can re-acquire a lock it already holds (prevents self-deadlock)

• **Memory visibility** — synchronized also ensures changes made in synchronized block visible to other threads

■ synchronized is blocking — thread waits indefinitely. Consider ReentrantLock with tryLock() for timeout-based locking

2.2 volatile Keyword

• Guarantees **visibility** — writes to volatile variable immediately visible to all threads (bypasses CPU cache)

• Guarantees **happens-before** — write to volatile happens-before any subsequent read of same variable

• Does **NOT** guarantee atomicity — volatile int counter; counter++ is still NOT atomic (3 operations: read, increment, write)

• Use volatile for: flags, status variables, singleton double-check

```
private volatile boolean running = true; // visibility guaranteed
// Thread 1: while(running) { ... }
// Thread 2: running = false; // visible to Thread 1 immediately
```

volatile vs synchronized

	volatile	synchronized
Visibility	Yes	Yes
Atomicity	No — only single read/write is atomic	Yes — entire block is atomic
Blocking	No — non-blocking	Yes — threads wait for lock
Use for	Simple flags, status, single-variable updates	Compound operations, critical sections
Performance	Faster — no lock	Slower — lock acquisition overhead

2.3 wait() / notify() / notifyAll()

• Defined on **Object** — must be called inside synchronized block on the same monitor

• **wait()** — releases lock and suspends thread until notified. Goes to WAITING state

• **notify()** — wakes one arbitrary waiting thread. Thread must reacquire lock before continuing

• **notifyAll()** — wakes all waiting threads. All compete for lock. Safer — prefer over notify()

• **Always use wait() in a loop** — spurious wakeups can occur

```
synchronized(lock) {
while(!condition) { // loop — not if! spurious wakeups
lock.wait(); // releases lock, waits
}
```

```
// condition is now true — do work
}
// In another thread:
synchronized(lock) { condition=true; lock.notifyAll(); }
```

✔ wait/notify is low-level. Prefer BlockingQueue or Condition (with ReentrantLock) in production code

2.4 Java Memory Model (JMM) & Happens-Before

- JMM defines rules for how threads interact through memory — what changes are visible to which threads
- **Happens-Before** — if A happens-before B, then A's effects are visible to B
- **Key happens-before rules:**
  - Program order: each action in a thread happens-before every subsequent action in same thread
  - Monitor lock: unlock of a monitor happens-before every subsequent lock of that monitor
  - Volatile: write to volatile field happens-before every subsequent read of that field
  - Thread start: Thread.start() happens-before any action in the started thread
  - Thread join: all actions in a thread happen-before Thread.join() returns
- **Without happens-before** — compiler/CPU/JVM may reorder instructions, cache values — leading to visibility bugs

■ Without synchronisation, there is NO guarantee other threads will see your thread's writes — even to simple booleans

3. java.util.concurrent.locks

3.1 ReentrantLock

- Explicit lock — more features than synchronized. Same reentrant semantics
- **tryLock()** — non-blocking attempt. tryLock(timeout, unit) — wait with timeout
- **lockInterruptibly()** — acquire lock but respond to interrupt
- **Fair lock** — new ReentrantLock(true) — FIFO order for waiting threads (slower but fair)
- **MUST unlock in finally block** — if exception occurs, lock must still be released

```
ReentrantLock lock = new ReentrantLock();
lock.lock();
try {
// critical section
} finally {
lock.unlock(); // ALWAYS in finally
}
```

```
// tryLock — no deadlock risk
if (lock.tryLock(500, TimeUnit.MILLISECONDS)) {
try { doWork(); } finally { lock.unlock(); }
} else { handleTimeout(); }
```

3.2 ReentrantReadWriteLock

- Separate read and write locks. Multiple threads can hold read lock simultaneously; write lock is exclusive
- **Read lock** — shared: multiple readers allowed concurrently (as long as no writer)
- **Write lock** — exclusive: only one writer; blocks all readers and other writers
- Use for: read-heavy data (config, cache) where writes are infrequent

```
ReadWriteLock rwLock = new ReentrantReadWriteLock();
Lock readLock = rwLock.readLock();
Lock writeLock = rwLock.writeLock();
// Reader: readLock.lock(); try{ read(); } finally{ readLock.unlock(); }
// Writer: writeLock.lock(); try{ write(); } finally{ writeLock.unlock(); }
```

3.3 StampedLock (Java 8+)

• More scalable than ReadWriteLock. Supports optimistic reads — no lock acquired, validate afterward

• **Optimistic read** — try reading without acquiring lock; validate if write occurred during read

• NOT reentrant. If validation fails, fall back to full read lock

```
StampedLock sl = new StampedLock();
long stamp = sl.tryOptimisticRead(); // no lock
double x = this.x; double y = this.y;
if (!sl.validate(stamp)) { // check if write happened
stamp = sl.readLock(); // fall back to read lock
try { x=this.x; y=this.y; } finally { sl.unlockRead(stamp); }
}
```

3.4 Condition (with ReentrantLock)

• Replacement for Object.wait/notify — more control with multiple conditions per lock

• **await()** — releases lock and waits (like wait()). **signal()**/**signalAll()** — like notify/notifyAll

```
ReentrantLock lock = new ReentrantLock();
Condition notFull = lock.newCondition();
Condition notEmpty = lock.newCondition();
// Producer: lock.lock(); while(full) notFull.await(); add(); notEmpty.signal(); lock.unlock();
// Consumer: lock.lock(); while(empty) notEmpty.await(); remove(); notFull.signal();
lock.unlock();
```

3.5 synchronized vs ReentrantLock

Feature	synchronized	ReentrantLock
Syntax	Keyword — implicit	Explicit lock/unlock
Release	Automatic (end of block)	Must call unlock() in finally
tryLock	No	Yes — non-blocking or with timeout
Interruptible	No — waits indefinitely	Yes — lockInterruptibly()
Fairness	No (non-fair)	Optional: new ReentrantLock(true)
Condition vars	One (wait/notify)	Multiple via newCondition()
Performance	Good (JVM optimised)	Slightly more overhead, more flexible
Use when	Simple locking, readability	Need tryLock, timeout, conditions, fair ordering

4. Atomic Classes (java.util.concurrent.atomic)

• Lock-free thread-safe operations using **CAS (Compare-And-Swap)** CPU instruction

• CAS: compare value in memory with expected; if equal, replace with new value — all atomically

• Much faster than synchronized for simple operations — no blocking

Class	Use For	Key Methods
AtomicInteger	Thread-safe int counter	get, set, incrementAndGet, decrementAndGet, addAndGet, compareAndSet, getAndUpdate
AtomicLong	Thread-safe long counter	Same as AtomicInteger
AtomicBoolean	Thread-safe boolean flag	get, set, compareAndSet, getAndSet

AtomicReference	Thread-safe object reference update	get, set, compareAndSet, updateAndGet, accumulateAndGet
AtomicIntegerArray	Thread-safe array of ints	get(i), set(i,v), incrementAndGet(i), compareAndSet(i,e,u)
AtomicStampedReference	Atomic ref + stamp — solves ABA problem	compareAndSet(expRef, newRef, expStamp, newStamp)
LongAdder	High-throughput counter — better than AtomicLong under contention	increment, add, sum, reset. Use for counters/statistics
LongAccumulator	Generalised LongAdder with custom accumulator function	accumulate(x), get

```
AtomicInteger counter = new AtomicInteger(0);
counter.incrementAndGet(); // thread-safe ++
counter.compareAndSet(5, 10); // if val==5, set to 10 — returns true/false
counter.updateAndGet(x -> x * 2); // apply function atomically
```

• **ABA Problem** — CAS only checks value, not history. A→B→A looks unchanged to CAS. Fix: AtomicStampedReference adds a version stamp

• **LongAdder vs AtomicLong** — LongAdder maintains separate cells per thread under contention, then sums. Better throughput; slower for single reads

5. Executor Framework

5.1 Why Executors?

• Creating a new Thread per task is expensive. Thread pools reuse threads — better resource management

• ExecutorService manages thread lifecycle — submit tasks, shutdown gracefully, monitor progress

5.2 Executors Factory Methods

Factory Method	Pool Type	Description	Use For
newFixedThreadPool(n)	Fixed, n threads	Always n threads alive. Queue: LinkedBlockingQueue (unbounded)	CPU-bound tasks with known parallelism
newCachedThreadPool()	Unbounded	Creates threads as needed, reuses idle ones. Idle threads die after 60s	Many short-lived async tasks
newSingleThreadExecutor()	1 thread	Single thread, tasks execute sequentially in submission order	Sequential task processing, ordering guaranteed
newScheduledThreadPool(n)	Fixed, scheduled	Schedule tasks with delay or fixed rate/interval	Periodic tasks, cron-like scheduling
newWorkStealingPool()	ForkJoinPool	Parallelism = CPU cores. Work-stealing — idle threads steal from busy ones	Compute-intensive, recursive tasks
newVirtualThreadPerTaskExecutor()	Virtual threads	Java 21 (Project Loom). Creates one virtual thread per task — scales to millions	IO-bound tasks in Java 21+

■ newCachedThreadPool can create unbounded threads — risk of OOM under heavy load. newFixedThreadPool with unbounded queue can cause queue memory issues. Prefer explicit ThreadPoolExecutor in production

5.3 ThreadPoolExecutor — Production Configuration

• Direct constructor gives full control over all pool parameters

Parameter	Description	Tuning Guidance
corePoolSize	Threads always kept alive even if idle	CPU tasks: #cores. IO tasks: higher (2-4x cores)
maximumPoolSize	Max threads allowed — triggers only when queue full	Cap to prevent resource exhaustion
keepAliveTime	Non-core threads idle this long before termination	60s typical
workQueue	Queue for tasks when all core threads busy	ArrayBlockingQueue(bounded), LinkedBlockingQueue(unbounded), SynchronousQueue
threadFactory	Creates new threads — use for naming/daemon config	Always name threads for debugging
rejectedHandler	Policy when queue full AND maxPoolSize reached	AbortPolicy(throw), CallerRunsPolicy(slow down producer), DiscardPolicy, DiscardOldestPolicy

```
ThreadPoolExecutor executor = new ThreadPoolExecutor(
4, // corePoolSize
8, // maximumPoolSize
60L, TimeUnit.SECONDS, // keepAliveTime
new ArrayBlockingQueue<>(100), // bounded queue — back-pressure
new ThreadFactory() { ... }, // named threads
new CallerRunsPolicy() // slow producer when full
);
```

5.4 ExecutorService Key Methods

Method	Description	Returns
submit(Callable)	Submit task that returns result	Future
submit(Runnable)	Submit task with no result	Future
execute(Runnable)	Submit fire-and-forget task	void
invokeAll(tasks)	Submit all, wait for all to complete	List>
invokeAny(tasks)	Submit all, return result of first to complete, cancel others	V
shutdown()	Stop accepting new tasks; finish submitted tasks	void
shutdownNow()	Attempt to stop all tasks; returns pending tasks	List
awaitTermination(t,unit)	Block until shutdown complete or timeout	boolean
isShutdown()	True after shutdown() called	boolean
isTerminated()	True after all tasks finished post-shutdown	boolean

```
// Graceful shutdown pattern
executor.shutdown();
if (!executor.awaitTermination(60, TimeUnit.SECONDS)) {
executor.shutdownNow();
}
```

5.5 Future & Callable

- **Callable** — like Runnable but returns V and can throw checked exception
- **Future** — handle to async result. get() blocks until result ready

Method	Description
get()	Block indefinitely until result. Throws ExecutionException (wraps task exception), InterruptedException
get(timeout, unit)	Block with timeout. Throws TimeoutException if not done in time
isDone()	True if task completed (normally, exceptionally, or cancelled)
isCancelled()	True if task was cancelled before completing
cancel(mayInterrupt)	Attempt to cancel. If running, mayInterrupt=true sends interrupt signal

6. CompletableFuture (Java 8+)

- Non-blocking async composition. Chain dependent async tasks without blocking threads
- Implements both Future and CompletionStage
- Runs on **ForkJoinPool.commonPool()** by default — or specify custom Executor

6.1 Creating CompletableFuture

```
// Async with no result
CompletableFuture f1 = CompletableFuture.runAsync(() -> doWork());
// Async with result
CompletableFuture f2 = CompletableFuture.supplyAsync(() -> fetchData());
// Completed immediately with value
CompletableFuture f3 = CompletableFuture.completedFuture("done");
```

6.2 Key CompletableFuture Methods

Method	Description	Async variant
thenApply(fn)	Transform result: T → U (like Stream.map)	thenApplyAsync(fn)
thenAccept(consumer)	Consume result, return void	thenAcceptAsync(consumer)
thenRun(runnable)	Run after completion, ignore result	thenRunAsync(runnable)
thenCompose(fn)	Chain — fn returns CompletableFuture (like flatMap)	thenComposeAsync(fn)
thenCombine(cf, fn)	When both this and cf complete, combine results	thenCombineAsync
exceptionally(fn)	Handle exception — provide fallback value	—
handle((res,ex) -> ...)	Handle both result and exception in one step	handleAsync
whenComplete((r,e)->...)	Callback on completion — does not transform value	whenCompleteAsync
allOf(cf1,cf2,...)	Wait for all CFs to complete (static method)	—
anyOf(cf1,cf2,...)	Complete when any CF completes (static method)	—
join()	Like get() but throws unchecked CompletionException	—
complete(value)	Manually complete with value (if not already done)	—
completeExceptionally(ex)	Manually complete with exception	—

```
// Pipeline example
CompletableFuture.supplyAsync(() -> fetchUser(id)) // async fetch
.thenApply(user -> enrichUser(user)) // transform
```

```
.thenCompose(user -> fetchOrders(user.getId())) // chain another async
.thenAccept(orders -> System.out.println(orders)) // consume
.exceptionally(ex -> { log(ex); return null; }); // handle error

// Wait for multiple
CompletableFuture f1 = supplyAsync(() -> callServiceA());
CompletableFuture f2 = supplyAsync(() -> callServiceB());
CompletableFuture.allOf(f1, f2).join(); // wait for both
String a = f1.join(); String b = f2.join();
```

✔ thenApply() vs thenCompose(): thenApply wraps result in CF — CF>. thenCompose flattens — CF. Same as map vs flatMap

7. Synchronisers (java.util.concurrent)

7.1 CountDownLatch

• One-time countdown gate. Initialised with count. Threads await() until count reaches 0 via countDown()

• **Cannot be reset** — single use. Use CyclicBarrier for repeatable barriers

• Use: wait for N services to start; wait for N tasks to complete; test coordination

```
CountDownLatch latch = new CountDownLatch(3); // 3 services must start
// Each service thread: doStartup(); latch.countDown();
// Main thread: latch.await(); // blocks until count = 0
latch.await(10, TimeUnit.SECONDS); // with timeout
```

7.2 CyclicBarrier

• Reusable barrier — N threads wait until all N reach the barrier, then all proceed together

• **Can be reset** — CyclicBarrier.reset(). Optional barrierAction runs when all threads arrive

• Use: parallel computation phases — compute in parallel, merge results, repeat

```
CyclicBarrier barrier = new CyclicBarrier(3, () -> System.out.println("Phase done"));
// Each of 3 threads: doPhaseWork(); barrier.await(); // wait for others
// When all 3 call await(), barrierAction runs, then all 3 proceed
```

7.3 Semaphore

• Controls access to a pool of resources. Maintains a count of permits

• **acquire()** — get a permit (blocks if 0 permits). **release()** — return a permit

• Semaphore(1) behaves like a mutex but can be released by different thread (unlike lock)

• Use: rate limiting, DB connection pools, throttling concurrent requests

```
Semaphore semaphore = new Semaphore(3); // max 3 concurrent access
semaphore.acquire(); // get permit — blocks if none available
try { accessResource(); } finally { semaphore.release(); }
semaphore.tryAcquire(500, TimeUnit.MILLISECONDS); // with timeout
```

7.4 Phaser (Java 7+)

• More flexible than CyclicBarrier — dynamic party registration, multiple phases

• Parties can register/deregister dynamically. arriveAndAwaitAdvance() = arrive + wait

• Use: multi-phase parallel algorithms with varying participant counts

```
Phaser phaser = new Phaser(3); // 3 parties
// Thread: doWork(); phaser.arriveAndAwaitAdvance(); // arrive and wait for others
phaser.register(); // add a party dynamically
phaser.arriveAndDeregister(); // done — remove from future phases
```

7.5 Exchanger

• Two threads exchange objects at a synchronisation point — each gets what the other gave

• Use: genetic algorithms (swap populations), data pipeline handoffs

```
Exchanger> exchanger = new Exchanger<>();
// Thread 1: List received = exchanger.exchange(myList);
// Thread 2: List received = exchanger.exchange(myList);
```

7.6 Synchronisers Comparison

Synchroniser	Parties	Reusable	Direction	Key Use Case
CountDownLatch	N→0	No	N threads signal 1	Wait for N events — startup gates, test coordination
CyclicBarrier	N wait	Yes	N threads meet N	Parallel phased computation — all meet at checkpoint
Semaphore	N permits	Yes	N concurrent access	Resource pools, rate limiting, throttling
Phaser	Dynamic	Yes	Multi-phase	Multi-phase tasks with dynamic participant count
Exchanger	2	Yes	Bidirectional swap	Two-thread data exchange at sync point

8. ThreadLocal

• Each thread has its own isolated copy of a variable — no sharing, no synchronisation needed

• Common uses: per-thread context (user session, request ID, DB connection, transaction)

• **MUST call remove()** in thread pools — thread pool threads are reused, ThreadLocal from previous task persists = memory leak and data leak

```
private static final ThreadLocal userId = new ThreadLocal<>();
// Set in request filter: userId.set(request.getUserId());
// Get anywhere in same thread: userId.get();
// MUST clean up: userId.remove(); // in finally block

// InheritableThreadLocal — child thread inherits parent's value
ThreadLocal tl = new InheritableThreadLocal<>();

// Java 20+ ScopedValue (preview) — better alternative to ThreadLocal
// Immutable, no need to remove, works with virtual threads
```

■ In Spring web apps, SecurityContextHolder and RequestContextHolder use ThreadLocal internally. Async methods break context propagation unless configured explicitly

9. Common Concurrency Problems

9.1 Race Condition

• Multiple threads access shared data concurrently, and outcome depends on thread scheduling order

• Classic example: i++ is NOT atomic (read-modify-write — 3 operations)

```
// Race condition — count can be lost
int count = 0; // Thread 1: read(0), +1, write(1)
// Thread 2: read(0), +1, write(1) — both write 1, not 2
// Fix: synchronized, AtomicInteger, or volatile (for simple flags)
```

9.2 Deadlock

- Two or more threads wait for each other's locks — circular dependency — none can proceed
- **Four necessary conditions (Coffman):** Mutual exclusion, Hold and wait, No preemption, Circular wait
- **Prevention strategies:**
  - **Lock ordering** — always acquire locks in same order across all threads
  - **tryLock with timeout** — avoid indefinite wait (ReentrantLock.tryLock())
  - **Avoid nested locks** — minimise holding multiple locks simultaneously
  - **Lock hierarchy** — number locks and always acquire in ascending order

```
// Deadlock example
Thread 1: lock(A); lock(B); // holds A, waits for B
Thread 2: lock(B); lock(A); // holds B, waits for A → deadlock
// Fix: both threads lock in same order: lock(A) then lock(B)
```

9.3 Livelock

- Threads are active but continuously responding to each other — no progress. Like two people in a corridor stepping aside simultaneously, forever
- Example: two threads keep retrying and backing off — neither makes progress
- Fix: add randomisation to backoff interval

9.4 Starvation

- Thread never gets CPU time — higher-priority threads always preempt or lock holder never releases
- Fix: fair locks (ReentrantLock(true)), avoid long critical sections, thread priority normalisation

9.5 Visibility Problem

- One thread's writes to shared variable not visible to other threads (cached in CPU registers/L1 cache)
- Fix: volatile, synchronized, Atomic classes — all establish happens-before

9.6 Double-Checked Locking (Singleton Pattern)

- Classic example combining volatile + synchronised correctly

```
public class Singleton {
private static volatile Singleton instance; // volatile essential
private Singleton() {}
public static Singleton getInstance() {
if (instance == null) { // first check (no lock)
synchronized(Singleton.class) {
if (instance == null) { // second check (with lock)
instance = new Singleton();
}
}
}
return instance;
}
}
// Better: use enum singleton or static holder pattern
```

9.7 Detecting and Analysing Deadlocks

- **jstack <pid>** — thread dump showing deadlock cycles
- **VisualVM / JConsole** — GUI thread dump and deadlock detection

• **ThreadMXBean** — programmatic deadlock detection via ManagementFactory.getThreadMXBean().findDeadlockedThreads()

10. Virtual Threads (Project Loom — Java 21)

• Lightweight threads managed by JVM, not OS. Can create **millions** — OS threads limited to thousands

• Mounted on carrier (platform) threads from ForkJoinPool. When virtual thread blocks (IO), it unmounts — carrier thread freed

• Designed for **IO-bound, high-concurrency** server workloads — not for CPU-bound tasks

• Same Thread API — existing code works with minimal changes

• Spring Boot 3.2+ supports virtual threads — enable with spring.threads.virtual.enabled=true

```
// Create virtual thread
Thread vt = Thread.ofVirtual().start(() -> handleRequest());
// Executor backed by virtual threads
ExecutorService exec = Executors.newVirtualThreadPerTaskExecutor();
exec.submit(() -> callExternalAPI()); // millions of these possible
```

	Platform Thread	Virtual Thread
Managed by	OS	JVM
Stack	~1MB fixed OS stack	Small, grows dynamically (few KB)
Count	Thousands (limited by OS)	Millions (limited by heap)
Blocking IO	Blocks carrier OS thread	Unmounts from carrier — carrier freed for other work
Best for	CPU-bound with known parallelism	IO-bound, high-concurrency — HTTP calls, DB queries
ThreadLocal	Works normally	Works but use with care — pinning risk
Pinning risk	—	synchronized blocks pin virtual thread to carrier — replace with ReentrantLock

11. Frequently Asked & Tricky Interview Questions

Thread Basics

Q1. What is a thread? How is it different from a process?

→ Thread: lightweight unit of execution within a process. Shares heap with other threads; own stack. Process: own memory space, isolated. Threads faster to create and communicate but share risk — one crash can kill all

Q2. What are the ways to create a thread in Java?

→ 1) Extend Thread and override run(). 2) Implement Runnable (preferred). 3) Implement Callable for result-bearing tasks. 4) Use ExecutorService (production best practice). Lambda works for Runnable/Callable

Q3. What is the difference between start() and run()?

→ start(): creates new OS thread, calls run() on it. run(): just a method call on current thread — no new thread created. Always call start() for multithreading

Q4. What are the thread states in Java?

→ NEW (created, not started), RUNNABLE (running or ready), BLOCKED (waiting for monitor lock), WAITING (wait/join/park indefinitely), TIMED_WAITING (sleep/wait/join with timeout), TERMINATED (completed)

Q5. What is a daemon thread? Examples?

→ Background thread — JVM exits when only daemon threads remain. Set via setDaemon(true) before start(). Examples: GC thread, finalizer thread. User threads keep JVM alive

Q6. What does Thread.sleep() do? Does it release the lock?

→ Pauses current thread for given ms — goes to TIMED_WAITING. Does NOT release lock. Thread.sleep(0) yields CPU. Throws InterruptedException if interrupted while sleeping

Q7. What is Thread.join()? Use case?

→ Current thread waits for target thread to finish. join(timeout) — wait at most timeout. Use: main thread waiting for worker threads to complete before aggregating results

Synchronisation

Q8. What is a race condition? Example?

→ Multiple threads access shared data, outcome depends on scheduling. Classic: i++ is 3 ops (read, add, write) — two threads both read 0, both write 1 instead of 2. Fix: synchronized, AtomicInteger

Q9. What does synchronized guarantee?

→ Mutual exclusion (only one thread in block at a time) + visibility (changes visible to all threads after lock release). Acquires monitor lock on entry, releases on exit — even on exception

Q10. What is the difference between synchronized instance method and static method?

→ Instance method: locks on 'this' — each object has its own lock. Static method: locks on Class object (e.g., MyClass.class) — shared across all instances. Two different locks

Q11. What is volatile? When to use it?

→ Guarantees visibility — writes immediately visible to all threads. Does NOT guarantee atomicity. Use for: simple flags (boolean running), status variables, single-read/write variables. Not for compound ops like i++

Q12. volatile vs synchronized?

→ volatile: visibility only, non-blocking, single variable. synchronized: visibility + atomicity + mutual exclusion, blocking, entire block. Use volatile for flags; synchronized for compound operations

Q13. What is the happens-before relationship?

→ If A happens-before B, A's changes are guaranteed visible to B. Key rules: unlock→lock same monitor, volatile write→subsequent reads, thread start→actions in thread, thread actions→join returns

Q14. What is wait()? Difference from sleep()?

→ wait(): releases lock and suspends — must be called in synchronized block on same monitor. sleep(): does NOT release lock, pauses fixed time. wait() goes WAITING; sleep() goes TIMED_WAITING

Q15. Why use wait() in a while loop, not an if?

→ Spurious wakeups — thread can wake from wait() without notify() being called. while loop re-checks condition and goes back to wait if not met. if block would proceed with invalid condition

Q16. notify() vs notifyAll()?

→ notify(): wakes one arbitrary waiting thread (risk: wrong thread woken, not matching condition). notifyAll(): wakes all — they compete for lock, each checks condition. notifyAll() is safer

Locks & Atomic

Q17. ReentrantLock vs synchronized?

→ ReentrantLock: explicit unlock (must be in finally), tryLock with timeout, lockInterruptibly, fair ordering, multiple conditions. synchronized: simpler syntax, auto-release, JVM-optimised. Use RL when you need these extra features

Q18. What is a fair lock?

→ new ReentrantLock(true) — threads acquire lock in FIFO order. Prevents starvation. But lower throughput — threads can't barge in. Non-fair (default) allows barging — higher throughput, possible starvation

Q19. What is ReadWriteLock? When to use?

→ Separate read lock (shared — many concurrent readers) and write lock (exclusive). Use for read-heavy data where reads don't modify state — config, cache. Multiple readers + zero writers can proceed simultaneously

Q20. What is CAS? What is the ABA problem?

→ CAS (Compare-And-Swap): CPU instruction — compare memory value with expected; if equal, replace with new — atomically. ABA: value changes A→B→A between read and CAS — CAS succeeds but intermediate change missed. Fix:

AtomicStampedReference

Q21. AtomicInteger vs synchronized counter? When to use each?

→ AtomicInteger: lock-free CAS, faster under low contention, good for single variable. synchronized: heavier but needed for compound operations across multiple variables. LongAdder beats both under high contention

Q22. What is StampedLock? What is optimistic reading?

→ More scalable ReadWriteLock alternative. Optimistic read: read without lock, validate afterward — if write occurred during read, fall back to full read lock. Not reentrant

Executor Framework

Q23. What is the Executor framework? Why use it?

→ Thread pool management — submit tasks without managing thread lifecycle. Benefits: thread reuse (cheaper), bounded concurrency, task queuing, rejection policies, graceful shutdown

Q24. Difference between submit() and execute()?

→ execute(Runnable): fire-and-forget, void return, exceptions propagated via UncaughtExceptionHandler. submit(Runnable/Callable): returns Future — track completion, get result, handle exceptions via future.get()

Q25. What are the Executors factory methods and when to use each?

→ newFixedThreadPool(n): CPU-bound known parallelism. newCachedThreadPool(): many short-lived tasks. newSingleThreadExecutor(): sequential ordering. newScheduledThreadPool(n): periodic tasks. newVirtualThreadPerTaskExecutor(): IO-bound Java 21

Q26. What are ThreadPoolExecutor rejection policies?

→ AbortPolicy(default): throws RejectedExecutionException. CallerRunsPolicy: caller thread runs task — natural back-pressure. DiscardPolicy: silently drops task. DiscardOldestPolicy: drops oldest queued task

Q27. How to size a thread pool?

→ CPU-bound: N = number of CPU cores (Runtime.getRuntime().availableProcessors()). IO-bound: N = cores * (1 + wait_time/service_time) — much larger. Profile under real load for best results

Q28. What is Future.get()? What exceptions can it throw?

→ Blocks until result available. ExecutionException: task threw exception (getCause() to unwrap). InterruptedException: waiting thread was interrupted. TimeoutException: get(timeout,unit) timed out

CompletableFuture

Q29. What is CompletableFuture? How is it better than Future?

→ Non-blocking async composition. Chain operations without blocking: thenApply, thenCompose, exceptionally, allOf. Future requires blocking get(). CF enables reactive-style pipelines without reactive frameworks

Q30. thenApply() vs thenCompose()?

→ thenApply(fn): fn returns value T — wraps in CF. Use for sync transform. thenCompose(fn): fn returns CF — flattens to CF. Use when fn itself is async. Same as map vs flatMap in streams

Q31. How to handle exceptions in CompletableFuture?

→ exceptionally(fn): catches exception, returns fallback value. handle((result,ex)->...): handles both success and failure. whenComplete: callback but doesn't change value. always chain exception handling

Q32. allOf() vs anyOf()?

→ allOf(cf1,cf2...): returns CF that completes when ALL complete. anyOf(cf1,cf2...): returns CF when ANY completes (with that result). Both static. allOf result Void — use cf.join() to get individual results

Q33. What thread does CompletableFuture run on?

→ runAsync/supplyAsync: ForkJoinPool.commonPool() by default. Pass custom Executor as second arg to control. thenApply/thenCompose without Async: runs in completing thread (or caller if already done). Use thenApplyAsync for dedicated thread

Synchronisers & ThreadLocal

Q34. CountDownLatch vs CyclicBarrier?

→ CountDownLatch: one-time, N→0 countdown, different threads count down and one waits. CyclicBarrier: reusable, N threads all wait for each other at same point, optional action when all arrive

Q35. What is a Semaphore? Use cases?

→ Limits concurrent access to resource via permits. acquire() blocks if 0 permits; release() returns permit. Use: DB connection pools, rate limiting (10 concurrent requests max), throttling

Q36. What is ThreadLocal? Memory leak risk?

→ Per-thread variable — each thread has own copy. Use for request context, user sessions. Memory leak in thread pools: thread reused but ThreadLocal value from prev task persists. MUST call remove() in finally

Q37. What is InheritableThreadLocal?

→ Child thread inherits parent thread's ThreadLocal values at creation time. Changes in child not visible to parent and vice versa after creation

Concurrency Problems

Q38. What is a deadlock? Four conditions?

→ Mutual wait cycle — threads wait for each other's locks. Four conditions (all must hold): Mutual exclusion, Hold-and-wait, No preemption, Circular wait. Remove any one to prevent deadlock

Q39. How to prevent deadlock?

→ Lock ordering: always acquire in same order. tryLock with timeout: no indefinite wait. Avoid nested locks. Single lock where possible. Lock-free algorithms (Atomic classes)

Q40. Deadlock vs Livelock vs Starvation?

→ Deadlock: threads blocked, no progress (waiting for locks). Livelock: threads active but no progress (keep reacting to each other). Starvation: thread never gets CPU/lock due to others always taking priority

Q41. How do you detect a deadlock in production?

→ jstack prints thread dump — shows 'Found deadlock' with cycle. JConsole/VisualVM GUI view. ThreadMXBean.findDeadlockedThreads() programmatically. Also: thread dumps show BLOCKED threads

Q42. What is double-checked locking? Why is volatile needed?

→ Singleton: check null without lock, lock, check again. volatile needed because without it, JVM can reorder instance = new Singleton() — another thread sees partially constructed object before volatile write completes

Virtual Threads & Advanced

Q43. What are virtual threads (Java 21)?

→ Lightweight JVM-managed threads. Millions possible vs thousands OS threads. When virtual thread blocks on IO, JVM unmounts it from carrier thread — carrier freed. Perfect for IO-bound high-concurrency workloads

Q44. When NOT to use virtual threads?

→ CPU-bound tasks (no IO blocking — no benefit). Code with synchronized blocks holding locks during IO (pinning — carrier thread blocked). ThreadLocal with heavy state. Always measure first

Q45. What is thread pinning in virtual threads?

→ When virtual thread inside synchronized block does blocking IO, it pins to carrier thread — carrier cannot serve other virtual threads. Fix: replace synchronized with ReentrantLock

Q46. What is ForkJoinPool?

→ Work-stealing thread pool for recursive divide-and-conquer tasks. Idle threads steal tasks from busy threads' queues. Used by CompletableFuture, parallel streams, virtual threads. RecursiveTask / RecursiveAction

Q47. What is the difference between Callable and Runnable?

→ Runnable: void run(), no return, no checked exception. Callable: V call(), returns value, can throw checked exception. Use Callable with ExecutorService.submit() when you need a result

Q48. How does CompletableFuture differ from reactive programming (Project Reactor/RxJava)?

→ CF: async pipeline for single values, limited operators. Reactor/RxJava: streams of events, backpressure support, rich operator set, better for continuous data streams. CF good for composing a few async calls; Reactor for complex reactive systems

Q49. What is thread starvation in a thread pool?

→ All pool threads blocked waiting for tasks they submitted — deadlock-like. e.g., all 4 threads submit a task that waits for another task — but pool is full. Fix: use separate pools for dependent tasks

Q50. How would you implement a thread-safe counter? Options and trade-offs?

→ synchronized method: simple, safe, slow. volatile int: visible but not atomic. AtomicInteger: CAS, lock-free, fast. LongAdder: fastest under high contention, sum() has slight delay. Choose based on contention level and read/write ratio